

数据库管理系统课程大作业

项目完成报告

项目名称：基于 MongoDB 的问卷系统开发
阶段：第一阶段

课程名称	数据库管理系统
技术栈	Node.js + Express + MongoDB + EJS
数据库	MongoDB (Mongoose 建模)
提交内容	项目完整报告

说明：本报告依据当前项目代码结构、MongoDB 设计、页面路由与服务层逻辑整理而成。

摘要

本项目实现了一个基于 MongoDB 的简化在线问卷系统，面向《数据库管理系统》课程大作业的要求进行设计与开发。系统支持用户注册与登录、问卷创建与编辑、问卷发布与关闭、匿名或登录填写、条件跳转、答案校验、提交保存以及统计查看等核心功能。整个系统采用 Node.js、Express、EJS 与 MongoDB 进行实现，重点体现数据库建模能力、后端业务逻辑设计能力以及文档整理能力。

在数据设计上，系统没有把所有信息堆叠到一个集合中，而是按职责拆分为 users、surveys、responses 三个核心集合，并结合 connect-mongo 持久化 session。问卷主体、题目定义、校验规则与跳转规则统一存储在 surveys 文档中；问卷提交结果独立存储在 responses 集合中，从而避免单文档无限膨胀，便于后续统计、扩展与多次迭代。

在业务实现上，系统将“跳转逻辑”和“校验逻辑”都设计为数据驱动方式：跳转规则保存在题目级 jumpRules 中，填写时由服务层动态匹配；校验规则保存在 validation 结构中，由前端即时校验和后端最终校验共同保证数据质量。系统同时记录 visitedQuestionIds，以表示用户真正访问过的题目路径，确保复杂跳转问卷下的校验与提交结果保持一致。

总体来看，本项目完成了课程所要求的主要功能，并在代码组织、数据库设计和可扩展性方面进行了额外考虑，为后续需求变更和阶段二扩展保留了清晰的修改空间。

关键词：MongoDB；问卷系统；条件跳转；数据校验；统计分析

1. 项目目标

1.1 课题背景

在线问卷系统广泛应用于课程调查、活动报名、意见收集、信息登记等场景。此类系统虽然界面可以较为简单，但后端需要同时处理题目建模、访问控制、答案校验、提交保存、统计分析等多个环节，因此非常适合作为数据库管理系统课程的大作业课题。

本项目围绕“基于 MongoDB 的问卷系统开发”展开，要求在不依赖关系数据库和现成问卷框架的前提下，独立完成问卷系统的数据结构设计、后端程序实现、测试与文档编写，并在报告中清楚说明 MongoDB 的设计依据、条件跳转逻辑、校验设计以及 AI 辅助使用过程。

1.2 项目建设目标

- 实现一个可运行的在线问卷系统，完成从用户注册登录到问卷统计查看的完整闭环。
- 使用 MongoDB 作为唯一核心业务数据库，对用户、问卷、提交结果进行统一存储。
- 支持单选、多选、文本、数字四类题目，并为题目配置校验规则与跳转规则。
- 支持问卷的创建、编辑、发布、关闭、填写、提交、统计等完整业务流程。
- 保证系统结构清晰、代码职责明确，为后续需求扩展保留可持续修改空间。

1.3 与课程要求的对应关系

课程要求	本项目实现内容	说明
用户账号功能	注册、登录、登出、Session 持久化	登录后仅能管理本人问卷
问卷创建与管理	创建、编辑、发布、关闭、截止时间、公开链接	问卷状态分为 draft / published / closed
题目类型	单选、多选、文本、数字	题目与规则统一嵌入 Survey 文档
跳转功能	单选/多选/数字/默认跳转	跳转规则由数据库驱动，不写死在控制器中
填写问卷	匿名/登录填写、重复提交限制、提交保存	支持 visitedQuestionIds 路径记录
统计结果	整卷统计、单题统计、平均值	统计数据从 responses 自动汇总

从上述对应关系可以看出，本项目已覆盖课程给出的主要功能点，并进一步考虑了问卷版本号、匿名重复提交限制、访问路径落库等实现细节，以提升系统完整性与可维护性。

2. 系统设计

2.1 技术选型与原因

层次	技术/组件	选型原因
后端框架	Node.js + Express	路由清晰、开发效率高，适合课程项目快速实现业务流程。
数据库	MongoDB + Mongoose	适合表达题目、选项、校验、跳转的嵌套结构，并便于文档建模。
视图层	EJS + 原生 JavaScript	无需额外前后端分离部署，便于展示完整业务流程。
登录态管理	express-session + connect-mongo	对课程项目足够稳定，且能直接把 session 持久化到 MongoDB。

这种技术组合的优势在于：一方面开发门槛适中，便于快速完成课程要求；另一方面结构足够清晰，可以在报告中明确说明控制器、服务层与数据库之间的职责分工。

2.2 总体架构

系统采用“页面渲染 + 控制器 + 服务层 + MongoDB”的总体架构，主要调用链路如下：浏览器请求页面或表单接口 → Express 路由分发 → Controller 处理权限与流程控制 → Service 处理数据清洗、跳转判断、校验与统计 → Mongoose Model 与 MongoDB 读写。

浏览器 / 表单 → **Express Routes** → **Controllers** → **Services** → **MongoDB**

在这种架构下，控制器不直接承担复杂业务判断，而是将问卷构建、答案校验、跳转计算与统计汇总分别封装在独立服务中，从而降低耦合，提高后续修改的可控性。

2.3 模块划分

- (1) 用户模块负责注册、登录、登出、Session 维护以及页面权限控制。
- (2) 问卷管理模块负责创建、编辑、发布、关闭问卷，并生成公开访问链接。
- (3) 问卷填写模块负责展示题目、根据答案动态跳题、生成提交前摘要并保存结果。
- (4) 跳转与校验模块负责依据问卷配置执行动态跳转，并保证所有提交内容满足规则要求。
- (5) 统计分析模块负责整卷统计与单题统计，针对不同题型返回不同形式的汇总结果。

2.4 主要业务流程

(1) 注册 / 登录流程：用户提交用户名与密码后，后端校验基本合法性；注册时将密码使用 bcryptjs 进行哈希后保存为 passwordHash；登录成功后把 userId 写入 session，并在后续请求中通过中间件加载当前用户。

(2) 创建 / 编辑问卷流程：前端问卷构建器将问卷信息、题目数组、校验规则和跳转规则组装为 `builderPayload`；后端通过 `surveyBuilderService` 进行解析、清洗、补全 `questionId` / `optionId`，并统一写入 `surveys` 集合。

(3) 发布 / 关闭流程：问卷创建者可将状态切换为 `published` 或 `closed`；发布时记录 `publishedAt`，关闭时记录 `closedAt`；填写入口会先检查问卷状态与截止时间。

(4) 填写 / 提交流程：填写者通过 `/survey/:slug` 进入问卷页面；前端根据题目配置逐题显示，在每次点击“下一题”时先做即时校验，再依据 `jumpRules` 计算下一题；提交时把 `answersPayload` 发送到后端，由后端重新计算 `visitedQuestionIds` 并进行统一校验后再写库。

(5) 统计流程：统计页面从 `responses` 集合读取已提交结果，对单选题和多选题统计各选项次数，对数字题计算平均值，对文本题展示所有文本内容，并支持整卷与单题两个层级的统计查看。

2.5 项目目录与职责

目录 / 文件	职责说明
<code>src/app.js</code>	系统入口，负责中间件、session、路由和当前用户加载。
<code>src/models</code>	Mongoose 模型，定义 User、Survey、Response 数据结构。
<code>src/controllers</code>	处理页面跳转、权限判断和业务流程调度。
<code>src/services</code>	封装问卷构建、跳转判断、校验与统计逻辑。
<code>src/routes</code>	定义认证路由、问卷路由和公共页面路由。
<code>views</code>	EJS 页面模板，包括 <code>builder</code> 、 <code>fill</code> 、 <code>dashboard</code> 、 <code>stats</code> 等页面。
<code>public/js</code>	前端问卷构建器与问卷填写交互逻辑。

3. MongoDB 设计

3.1 为什么使用 MongoDB

首先，课程要求明确规定后端数据库必须使用 MongoDB；其次，问卷系统本身天然具有较强的文档型特征：一份问卷往往由问卷基础信息、题目数组、选项数组、校验规则、跳转规则等多层嵌套数据组成，如果强行拆分成大量关系表，会带来较多的连接关系与代码复杂度。

MongoDB 的文档模型更适合将“一份问卷”作为一个业务整体来读取和更新。对于本项目而言，创建者在编辑问卷时通常需要一次性获取完整问卷结构；填写者在进入问卷时也需要一次性拿到所有题目、选项和规则，因此题目定义与规则配置嵌入 `Survey` 文档的方式更自然，也更符合实际访问模式。

3.2 集合划分

集合	主要内容	设计目的
users	账号、密码哈希、创建时间	独立保存用户身份信息，便于权限控制。
surveys	问卷标题、设置、题目、校验规则、跳转规则	作为问卷业务主体，支持整体读取与整体编辑。
responses	每次提交的答案、访问路径、提交人信息	避免问卷文档膨胀，便于统计与重复提交判断。
sessions	用户会话信息	由 connect-mongo 存储，实现登录态持久化。

其中前三个集合属于系统核心业务集合，sessions 集合用于保存 express-session 的会话数据。虽然 sessions 不直接参与问卷业务统计，但它体现了 MongoDB 在整个系统中的统一使用方式。

3.3 users 集合设计

users 集合用于保存用户账号信息，字段相对简洁，但承担着身份认证和权限控制的基础功能。其主要字段包括：_id、username、passwordHash、createdAt、updatedAt。

字段	类型	约束	说明
username	String	唯一、必填、3~30 字符	用户登录名，用于查重与登录验证。
passwordHash	String	必填	使用 bcryptjs 哈希后的密码，不保存明文密码。
createdAt / updatedAt	Date	自动维护	记录用户创建和更新时间。

由于 username 被设置为唯一索引，系统可以在注册阶段直接阻止重名账号；passwordHash 的设计保证了即使数据库被查看，也不会直接暴露用户原始密码。

3.4 surveys 集合设计

surveys 集合是本项目最核心的集合，用于保存问卷主体、题目内容、校验规则、跳转规则以及状态信息。由于一份问卷通常需要被整体读取，因此 questions 嵌入在 surveys 中更符合访问特征。

字段组	典型字段	作用
基础信息	ownerId、title、description、slug	标识问卷所有者、标题、说明和公开访问链接。
状态信息	status、publishedAt、closedAt	描述问卷是否为草稿、已发布或已关闭。

问卷设置	allowAnonymous、allowMultipleSubmissions、deadlineAt	控制匿名填写、重复提交与截止时间。
版本控制	version	每次编辑保存后自动加 1，为后续迭代预留空间。
题目顺序	questionOrder	明确题目顺序，便于顺序跳转与重建路径。
题目数组	questions[]	保存题目标题、题型、必答、选项等核心配置。
校验规则	validation.text / number / multi	描述文本长度、数字范围、整数限制、多选数量限制等。

每个题目还包含 `jumpRules` 数组，用于描述不同回答条件下的目标跳转位置。这样设计的好处是，问卷的“结构”和“行为规则”都被保存为数据，而不是散落在代码分支中。

3.5 responses 集合设计

`responses` 集合用于保存每一次问卷提交的结果。系统并没有把所有回答直接嵌入到 `surveys` 中，而是将其设计为独立集合。这样可以避免单份问卷随着提交次数增加而不断膨胀，也有利于多人并发提交和后续统计分析。

字段	类型	说明
surveyId	ObjectId	对应哪一份问卷。
surveyTitleSnapshot	String	提交时保存问卷标题快照，减少统计展示时的依赖。
surveyVersion	Number	记录提交时所对应的问卷版本。
respondentId / respondentType	ObjectId / String	区分登录用户与匿名用户。
visitedQuestionIds	String[]	记录用户实际访问过的题目路径。
answers[]	数组	每项包含 <code>questionId</code> 、 <code>questionType</code> 、 <code>value</code> 。

`visitedQuestionIds` 是 `responses` 集合中的一个关键字段。由于问卷支持条件跳转，并不是所有题目都会被所有用户访问，因此只有记录真实访问路径，才能在后端只校验真正走到的题目，并为后续统计与调试保留依据。

3.6 索引与可扩展性设计

从当前代码可以看出，系统已经为几个高频字段建立了索引。例如 `User.username` 为唯一索引，`Survey.slug` 为唯一索引，`Survey.ownerId`、`Survey.status`、`Response.surveyId`、`Response.respondentId`、`Response.submittedAt` 等字段也被用于快速过滤与查询。

这些索引能够支撑当前项目中最常见的访问场景：注册登录、按 `slug` 打开问卷、按 `ownerId` 查询“我的问卷”、按 `surveyId` 汇总统计、按 `respondentId` 检查重复提交等。与此同

时, Survey.version、settings、validation、jumpRules 和 Response.visitedQuestionIds 等字段为第二阶段扩展保留了较好的兼容性。

3.7 与关系数据库方案的对比

如果使用关系数据库实现同样功能, 通常需要设计 surveys、questions、options、jump_rules、responses、answers 等多张表, 再通过外键维护它们之间的层级关系。在创建问卷、进入问卷填写页以及统计时, 往往需要多次 join 才能还原完整结构。

相比之下, 本项目中的题目、选项、校验与跳转规则层级明显, 且单份问卷通常会整体读取, 因此 MongoDB 的文档嵌入方式更贴近真实业务对象。对于课程项目规模而言, 这种设计在表达复杂结构和降低开发复杂度之间取得了较好的平衡。

4. 跳转设计

4.1 跳转需求分析

问卷并不总是严格按照题目编号顺序填写。在实际场景中, 用户对前题的回答会影响后续应当看到的题目。例如用户选择某种身份后, 后续只需要看到与该身份相关的问题; 又例如数字题可以根据年龄区间或评分区间进入不同分支。因此, 问卷系统需要支持可配置的条件跳转。

课程要求特别强调跳转逻辑不能写死在程序里, 而应当能够随着问卷配置变化而动态生效。本项目因此选择将跳转规则持久化到数据库中, 再由前端和后端共同依据这些规则计算下一题。

4.2 跳转规则的数据结构

在 Survey 模型中, 每个题目都可以拥有 jumpRules 数组。每条规则包含 priority、ruleType、value / values / min / max、targetQuestionId 等字段。其中 priority 用于控制匹配顺序, targetQuestionId 用于指定下一题的 questionId; 若目标值为 __SUBMIT__, 则表示跳转到提交阶段。

字段	示例	含义
priority	1	匹配优先级, 数字越小越先执行。
ruleType	single_equals	规则类型, 例如单选等于、多选包含、数字比较等。
value	option-a / 18	单值比较时使用。
values	["A", "B"]	多值比较时使用。
min / max	10 / 20	区间比较时使用。
targetQuestionId	question-5 / __SUBMIT__	命中规则后跳转的目标题目或提交页。

4.3 当前支持的跳转类型

- single_equals: 单选题答案等于某个 optionId 或选项文本时跳转;

- `multi_contains_any`: 多选题包含任意一个指定选项时跳转;
- `multi_contains_all`: 多选题同时包含多个指定选项时跳转;
- `number_gt` / `number_gte` / `number_lt` / `number_lte`: 数字题按大小关系跳转;
- `number_between`: 数字题落在指定区间内时跳转;
- `always`: 默认跳转, 用于兜底或直接进入提交阶段。

4.4 跳转执行流程

系统在前端填写页面与后端提交阶段都执行了相同思路的跳转计算。其核心过程如下: 首先根据当前题 `questionId` 找到题目对象; 然后获取当前答案; 接着按 `priority` 对 `jumpRules` 排序, 并从前到后依次匹配; 第一条命中的规则立即生效, 返回其 `targetQuestionId`; 如果没有任何规则命中, 则回退到 `questionOrder` 所定义的顺序下一题。

在前端 `public/js/finishSurvey.js` 中, 用户每点击一次“下一题”, 就会先收集当前答案、做即时校验, 再调用 `getNextQuestionId` 计算下一题; 如果返回值为空, 则说明应当进入提交前摘要页。

在后端 `submitSurvey` 中, 系统会根据 `answersPayload` 重新计算 `visitedQuestionIds`, 而不是直接相信前端传来的路径。这样可以避免用户绕过前端逻辑伪造提交路径, 也能确保真正落库的数据和问卷规则保持一致。

4.5 visitedQuestionIds 的设计意义

`visitedQuestionIds` 的作用不仅仅是“记录路径”, 它实际上同时解决了三个问题。第一, 它使后端只需要校验真正访问过的题目, 避免未经过的分支题目被误判为未填写。第二, 它为统计和调试提供了依据, 后续若要分析不同用户走过的分支路径, 可以直接从 `responses` 中读取。第三, 它为第二阶段扩展更复杂的跳转、分页或流程回溯预留了空间。

4.6 为什么这种设计更适合后续扩展

由于跳转是以规则数据的形式保存, 而不是散落在 `if/else` 代码中, 因此后续如果要增加新的 `ruleType`, 例如“文本匹配”“分值区间映射”或“跨页跳转”, 主要修改 `Survey Schema` 与匹配函数即可, 不需要重写整个问卷填写流程。这也符合课程中“第二阶段不得推倒重做”的要求。

5. 校验设计

5.1 校验目标

问卷系统的一个核心任务是防止非法数据进入数据库。如果不做校验, 用户可能提交空答案、越界数字、无效选项, 甚至直接伪造表单内容, 这会导致统计结果失真, 也会破坏系统的可靠性。因此, 本项目将校验划分为前端即时校验与后端统一校验两个层次。

5.2 前端校验

前端校验主要实现于 `public/js/fillSurvey.js` 中，作用是提升填写体验。用户在点击“下一题”时，页面会先读取当前题答案，并调用 `clientValidate` 进行本地判断；若存在问题，则直接显示错误提示，不进入下一题。

- 必答题为空时给出即时提示；
- 多选题根据 `exactSelected`、`minSelected`、`maxSelected` 给出数量限制提示；
- 文本题根据 `minLength` 和 `maxLength` 控制字数范围；
- 数字题检查是否为合法数字、是否要求整数、是否越界。

此外，前端页面还会在多选题下方展示“至少选择几项 / 最多选择几项 / 必须选择几项”等提示语，减少误填概率。

5.3 后端校验

后端校验由 `validationService.js` 统一处理，并在 `submitSurvey` 中对 `visitedQuestionIds` 对应的每一道题依次执行。与前端校验相比，后端校验是最终的、不可绕过的，因此它承担的是数据安全职责。

题型	校验内容	实现效果
single_choice	必答检查、optionId 合法性检查	防止空答和非法单选值写入数据库。
multi_choice	数组格式检查、选项合法性、最少/最多/恰好选择数检查	保证多选题数据结构与数量约束正确。
text	必答检查、最小长度、最大长度	控制文本题内容范围。
number	必答检查、合法数字、整数限制、最小值、最大值	防止非数字或超范围数值进入结果集。
整体提交	仅校验 <code>visitedQuestionIds</code> 对应题目	避免未访问分支题目误报错误。

这种“前端提升体验、后端保证安全”的分层校验方式较为稳妥。即使用户通过浏览器开发者工具手动修改表单，也无法绕过后端的最终判断。

5.4 与题目建模的关系

值得注意的是，本项目的校验不是硬编码在每个页面里，而是直接依赖 `questions[].validation` 中的数据配置。也就是说，问卷创建者在构建问卷时输入的最小长度、最大值、整数限制、多选数量限制等参数，都会在提交阶段被动态读取并执行。这样可以保证“问卷规则”与“校验逻辑”保持一致。

5.5 重复提交限制也是一种校验

除了题目答案本身，系统还对“是否允许重复提交”做了额外控制。当 allowMultipleSubmissions 为 false 时：如果填写者是登录用户，则后端按 surveyId + respondentId 查询是否已有提交记录；如果填写者是匿名访客，则通过 session 中的 anonymousSubmittedSurveyIds 进行限制。

这种设计虽然对匿名用户的限制粒度不如账号级限制精确，但对于课程项目阶段而言实现简单、逻辑清晰，并且能够体现系统对业务规则的完整性考虑。

6. AI 使用过程

6.1 使用原则

本项目在启动阶段使用了 AI 工具辅助整理需求和形成基础实现思路，但最终代码、字段设计、功能取舍、联调修正、错误提示补充和文档整理均经过人工筛选、修改与确认。

6.2 AI 使用记录

使用阶段	项目启动阶段
使用目的	梳理课程需求、确定 MongoDB 问卷系统的基本数据结构与代码组织方向
使用工具	ChatGPT
代表性提示词	现在你是一个数据库管理系统的专家，给我这个大作业的第一阶段的代码组织方向，具体要求看一下我发给你的doc文档，然后仔细阅读老师说的每一个要求，不要漏掉任何一个要求，一步步给我每个阶段的代码，并教我怎么做
使用阶段	项目修改阶段
使用目的	在碰到自己目前水平难以解决的问题时，寻求一些代码修改的帮助
使用工具	ChatGPT
代表性提示词	我明明已经发布了我的投票，但是为什么打开我的投票链接后说我404 not found；为什么我选择”没参加过“还是继续在Q2？然后我回到teacher_demo创造的问卷里，发现跳转规则好像没有被保存，不知道为什么？
使用阶段	数据提供阶段
使用目的	给出一些手工测试的数据，以此检验问卷系统构造是否过关
使用工具	ChatGPT

代表性提示词	老师还给了测试要求 必须编写测试用例。至少包括：创建问卷测试 添加题目测试 跳转逻辑测试 校验测试 提交问卷测试 统计测试。给我几个测试数据，最好能包含所有类别的问卷
--------	---

6.3 AI 带来的帮助

- 帮助快速梳理老师给出的功能点，减少需求遗漏；
- 帮助形成 users / surveys / responses 的初始集合划分思路；
- 帮助确定 Controller + Service + Model 的基础结构和给出基本代码框架，便于后续代码问题的调试和组织；
- 帮助搭建报告与文档的初始框架，提高整理效率。
- 帮助修改代码中存在的一些目前能力无法解决的问题，并提升自己对代码的理解。
- 帮助给出问卷的测试数据

6.4 人工修改与自主完成的部分

项目后续的大量工作并不是直接照搬 AI 结果，而是根据实际运行效果不断调整完成的。例如：登录错误提示、多选题选择提示、前端填写交互、重复提交限制、提交阶段重新计算 visitedQuestionIds、统计页面组织、文档内容与课程要求的对齐等，均需要结合当前代码结构和测试情况手工修正。

换言之，AI 在本项目中更多起到“起步辅助”和“思路参考”的作用，而不是最终成果的替代。最终提交版本的功能完整性和可运行性仍然依赖本人对需求、代码与数据库结构的理解。

6.5 对 AI 使用的反思

通过本项目可以看到，AI 能够显著提高原型搭建速度，但它给出的内容往往更偏向“通用正确”，在真实项目中仍然存在字段命名不统一、边界条件不充分、交互体验不够细致等问题。因此，在课程项目中合理使用 AI 的关键，不是让 AI 代替思考，而是把它当作一个加速器，然后由自己完成验证、筛选、修改和整合。

7. 测试结果

7.1 测试方式说明

本项目测试以手工功能测试为主，结合页面操作、数据库结果检查和统计页面查看三种方式进行。测试重点覆盖课程要求中的六类核心场景：创建问卷、添加题目、跳转逻辑、校验逻辑、提交问卷以及统计分析。详细步骤可在单独的测试用例文档中展开，项目完成报告中主要汇总测试结果。

7.2 主要测试结果汇总

序号	测试项目	测试内容	结果概述	结论
1	创建问卷	创建包含单选、多选、文本、数字题的问卷并保存	问卷成功写入 surveys, 自动生成 slug, 编辑页可再次打开	通过
2	题目与规则保存	设置多选数量限制、文本长度限制、数字范围限制	questions[] 与 validation 结构保存正确, 重新进入编辑页后配置仍存在	通过
3	跳转逻辑	测试单选跳转、多选包含跳转、数字区间跳转	填写时能按规则进入目标题目或直接进入提交阶段	通过
4	校验逻辑	测试必答为空、多选少选/多选过量、文本太短、数字越界等情况	前端可即时提示, 后端也会阻止非法提交	通过
5	提交与重复限制	测试登录用户与匿名用户的提交流程及重复提交限制	responses 写入成功; 当禁止重复提交时会给出阻止提示	通过
6	统计分析	查看整卷统计和单题统计	单选/多选统计数量正确, 数字题可显示平均值, 文本题可展示全部回答	通过

7.3 典型测试场景说明

(1) 跳转测试: 构造一份包含 5 道题的问卷, 在第 1 题中设置不同选项跳向不同题目, 填写时分别输入不同答案, 系统能够按照 jumpRules 的优先级正确命中目标题目。

(2) 校验测试: 针对多选题设置“至少选择 2 项, 最多选择 3 项”, 故意只选 1 项或超过 3 项时, 前端会立即提示错误; 即便绕过前端强行提交, 后端仍会返回“该题至少选择 2 个选项”或“该题最多选择 3 个选项”等信息。

(3) 重复提交测试: 当问卷设置 allowMultipleSubmissions = false 时, 已登录用户第二次提交同一问卷会被直接阻止; 匿名用户在同一会话内再次提交, 也会收到“重复提交受限”的提示。

(4) 统计测试: 对同一问卷提交多份回答后进入统计页, 系统能够按题型自动生成选项计数、文本列表和数字平均值, 说明 responses 集合的结构能够有效支持后续统计。

7.4 测试结论

综合测试结果可以看出，项目已实现从“创建问卷—发布问卷—填写问卷—提交结果—查看统计”的完整主流程，并且在跳转、校验、匿名填写和重复提交限制等方面达到预期效果。当前系统已经具备课程大作业第一阶段所要求的主要功能。

8. 遇到的问题

8.1 数据结构如何兼顾灵活性与可维护性

问卷系统中的题目类型、校验规则和跳转规则差异较大。如果把所有字段完全平铺，结构会显得混乱；如果拆成过多集合，又会导致读取问卷时需要大量关联查询。因此，如何在“灵活表达复杂结构”和“保持代码可维护性”之间取得平衡，是实现初期遇到的第一个问题。

8.2 条件跳转会打破顺序填写假设

一旦问卷支持跳转，就不能再简单地假设所有用户都会按固定顺序答完所有题。这样会引发两个连锁问题：第一，哪些题目应该被展示和校验；第二，提交结果中如何表示用户真实走过的路径。

8.3 前后端校验容易不一致

如果前端提示一套规则，后端又按另一套逻辑处理，就会出现“页面上看起来可以提交，但服务器又报错”的体验问题。因此，如何让题目规则、前端交互和后端校验尽量一致，也是开发过程中需要重点处理的问题。

8.4 匿名填写下的重复提交控制较难

登录用户可以通过 `respondentId` 直接识别身份，但匿名用户没有账号标识。如果课程又要求支持匿名填写和“可禁止重复提交”，系统就必须在缺少身份信息的情况下设计一个合理的限制方案。

8.5 统计功能必须兼顾不同题型

单选题、多选题、文本题和数字题的统计方式差别很大，尤其是数字题需要计算平均值，而文本题更适合展示原始内容。因此，如何为不同题型组织统一的数据输出结构，也是统计功能设计中的难点。

9. 如何解决

9.1 通过“核心集合拆分 + 问卷内部嵌入”解决结构问题

针对数据结构问题，本项目采用 `users`、`surveys`、`responses` 三个核心集合的划分方式：用户信息、问卷定义与提交结果分别独立存储；同时将题目、选项、校验规则和跳转规则嵌入

surveys 文档内部。这样既避免了“一个集合装所有数据”的混乱，也保留了整份问卷整体读取的优势。

9.2 通过 visitedQuestionIds 解决跳转后的校验与回溯问题

针对跳转问题，系统在提交阶段重新依据 answersPayload 计算 visitedQuestionIds，并把这一结果写入 responses。此举保证了后端只校验用户真正访问过的题目，同时让统计和调试能够追踪填写路径。前端填写页也按相同规则实时决定下一题，确保页面行为与后端逻辑一致。

9.3 通过“前端即时校验 + 后端统一校验”保证一致性

针对前后端校验不一致的问题，系统把题目限制统一建模到 questions[].validation 中，再由前端与后端分别读取同一组规则进行处理。前端负责提升交互体验，后端负责最终兜底，从而尽量减少逻辑分裂和重复维护成本。

9.4 通过 session 机制限制匿名重复提交

针对匿名填写无法直接识别用户身份的问题，项目采用 session 中的 anonymousSubmittedSurveyIds 记录匿名访客已提交过的问卷 ID。当 allowMultipleSubmissions = false 时，系统会优先检查该列表并阻止重复提交。虽然这种方式仍可能被清理浏览器会话绕过，但对于课程项目阶段已经能够较好体现业务规则控制。

9.5 通过服务层封装统一统计接口

针对不同题型统计方式不一致的问题，项目将统计逻辑集中封装到 statsService 中。服务层先从 responses 集合读取某问卷全部提交，再按题目类型分别生成 optionCounts、allValues、average 等字段，最后交由页面统一渲染。这样既保证了统计逻辑清晰，也使单题统计和整卷统计可以共享同一套基础数据。

9.6 通过版本号和状态字段为后续扩展保留空间

除了当前功能，本项目还保留了若干有利于后续扩展的设计，例如 Survey.version、status、settings、jumpRules、visitedQuestionIds 等字段。这意味着在第二阶段如果新增题型、增加统计方式或细化流程控制，系统可以在现有基础上继续扩展，而不必推倒重做。

10. 总结

综上所述，本项目围绕“基于 MongoDB 的问卷系统开发”这一目标，完成了用户注册登录、问卷创建与管理、条件跳转、答案校验、问卷填写、提交保存和统计分析等完整功能流程。项目在 MongoDB 数据设计上充分利用了文档模型的优势，使一份问卷能够以相对自然的结构保存题目、校验规则与跳转规则；在后端实现上，则通过控制器、服务层和模型的分工保证了逻辑清晰。

更重要的是，本项目并没有只停留在“能运行”的层面，而是针对重复提交限制、匿名填写、访问路径记录、版本控制等细节做了补充设计，使系统更符合真实应用场景，也更有利于后续阶段的增量修改。

因此，本项目已经较好地完成了课程要求，能够作为第一阶段的完整成果提交，并为后续需求变更打下稳定基础。

PS:代码仓库网站: <https://github.com/Yvon-ld/Database-Lab>